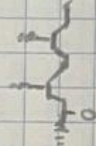
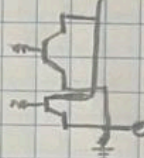
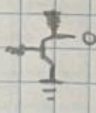
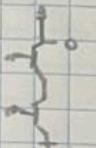
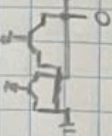
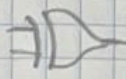
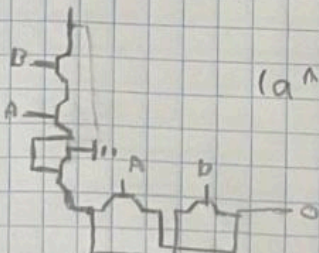


Introducción

Jorge Abraham Moran Encina

¿Qué es una FPGA? Como se relaciona con el lenguaje de descripción de hardware VERILOG

- Field-Programmable Gate Array es un circuito integrado reconfigurable tal como dice su nombre, esta compuesto por bloques lógicos y redes de interconexión que se pueden configurar para implementar diferentes circuitos digitales.
- Con verilog se describe como se comporten los componentes, entradas y salidas. Se escribe el código, se simula el diseño, se sintetiza el código y se implementa en la FPGA.

	Simbología	Tablas de verdad	Arreglo	Verilog															
AND		<table><tr><th>A</th><th>B</th><th>C</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	A	B	C	0	0	0	0	1	0	1	0	0	1	1	1		$a \& b$
A	B	C																	
0	0	0																	
0	1	0																	
1	0	0																	
1	1	1																	
OR		<table><tr><th>A</th><th>B</th><th>C</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	A	B	C	0	0	0	0	1	1	1	0	1	1	1	1		$a b$
A	B	C																	
0	0	0																	
0	1	1																	
1	0	1																	
1	1	1																	
NOT		<table><tr><th>A</th><th>B</th></tr><tr><td>0</td><td>1</td></tr><tr><td>1</td><td>0</td></tr></table>	A	B	0	1	1	0		$\sim a$									
A	B																		
0	1																		
1	0																		
NAND		<table><tr><th>A</th><th>B</th><th>C</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	A	B	C	0	0	1	0	1	1	1	0	1	1	1	0		$\sim (a \& b)$
A	B	C																	
0	0	1																	
0	1	1																	
1	0	1																	
1	1	0																	
NOR		<table><tr><th>A</th><th>B</th><th>C</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	A	B	C	0	0	1	0	1	0	1	0	0	1	1	0		$\sim (a b)$
A	B	C																	
0	0	1																	
0	1	0																	
1	0	0																	
1	1	0																	
XOR		<table><tr><th>A</th><th>B</th><th>C</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	A	B	C	0	0	0	0	1	1	1	0	1	1	1	0		$(a \wedge b)$
A	B	C																	
0	0	0																	
0	1	1																	
1	0	1																	
1	1	0																	

Objetivos

Esta actividad es para que se tenga la oportunidad de corroborar la descarga e instalación correcta del compilador y simulador de HDL a utilizar durante el semestre, cualquier duda o error con el SW debe ser resuelto en durante esta práctica.

Así como comprender e investigar la sintaxis de cada uno de los operadores de lógica combinacional en el HDL elegido (Verilog o VHDL).

Las operaciones lógicas a implementar son las 7 compuertas lógicas principales **AND, NAND, OR, NOR, NOT, XOR, XNOR**, siendo la implementación a nivel de 1 bit, es decir operaciones lógicas de dos bits de entrada y uno de salida en cada uno (excepto la compuerta NOT) Éstas deben ser implementadas todas en un solo proyecto y en un solo módulo (un solo archivo ".v") con dos entradas y siete salidas.

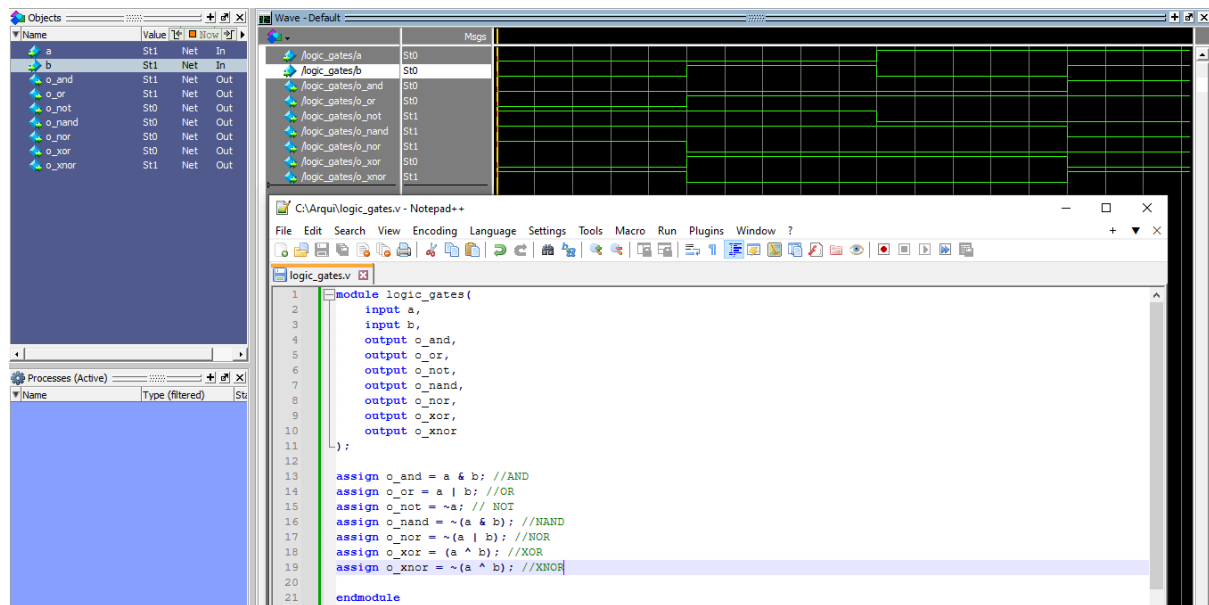
Desarrollo

Aquí podemos observar el código, tal como se pidió se creó un módulo el cual tiene dos entradas y siete salidas, cada una correspondiente a una compuerta lógica.

```
module logic_gates(
    input a,
    input b,
    output o_and,
    output o_or,
    output o_not,
    output o_nand,
    output o_nor,
    output o_xor,
    output o_xnor
);

    assign o_and = a & b; //AND
    assign o_or = a | b; //OR
    assign o_not = ~a; // NOT
    assign o_nand = ~(a & b); //NAND
    assign o_nor = ~(a | b); //NOR
    assign o_xor = (a ^ b); //XOR
    assign o_xnor = ~(a ^ b); //XNOR

endmodule
```



Conclusiones

Esta práctica estuvo sencilla, fue para poder familiarizarnos más con las aplicaciones que vamos a utilizar y para aprender la sintaxis básica de las compuertas lógicas principales.